Module 3: Data Link Layer and Wireless Networks

Module 3: Data Link Layer and Wireless Networks

Data-Link Layer

- Data Link Control (DLC)
- Multiple Access Protocols (MAC)
- Link-Layer Addressing
- Ethernet Protocol
- Connecting Devices

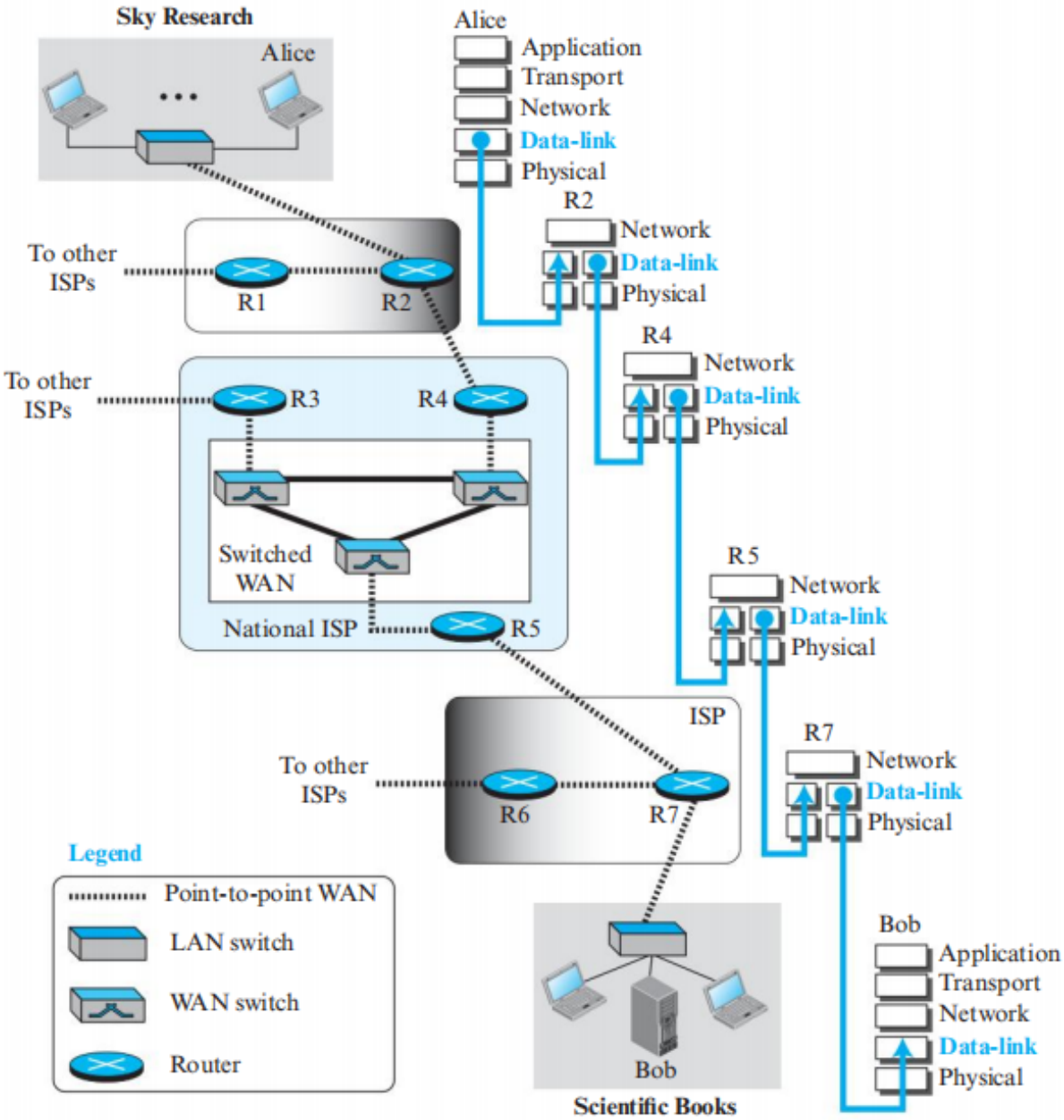
Wireless Networks

- Wireless LANs
- Mobile IP

Hands-On

- Datalink Provider Interface
- SOCK_PACKET
- PF_PACKET

Figure 5.1 *Communication at the data-link layer*



Why Do We Need the Data-Link Layer?

- The **network layer** provides **host-to-host communication**, but a datagram may need to cross several interconnected networks.
- **Routers** connect these networks and forward the datagram from one network to the next.
- At the **data-link layer**, communication occurs separately over each link between two directly connected devices.

Network Layer

Host-to-Host

Alice $\longrightarrow$ Bob

End-to-end communication

Data-Link Layer

Node-to-Node

Alice $\longrightarrow$ R2 $\longrightarrow$ R4 $\longrightarrow$ R7 $\longrightarrow$ Bob

Separate communication on each link

Key Idea

A packet travels **host-to-host**, but each hop is handled **link-by-link** by the data-link layer.

Data-Link Layer: Nodes, Links & Sublayers

1. Nodes and Links

- At the **application, transport and network layers**, communication is **end-to-end**; at the data-link layer it is **node-to-node**.
- A packet may cross several **LANs and WANs**, connected by routers, before reaching its destination.
- **Nodes** = source host, destination host and intermediate routers; **links** = networks connecting adjacent nodes.

Example:

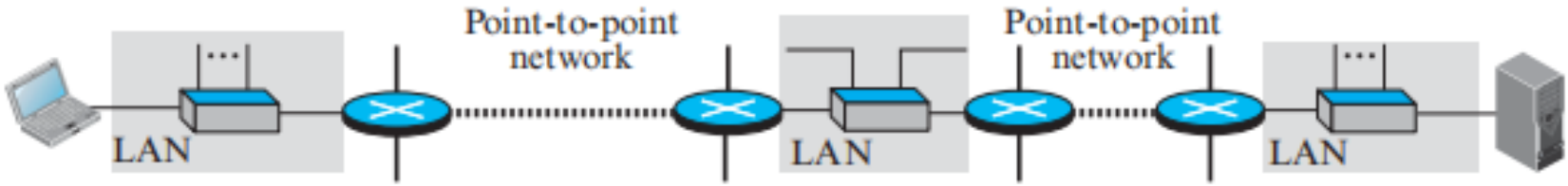
Host $\rightarrow$ R1 $\rightarrow$ R2 $\rightarrow$ R3 $\rightarrow$ Host

Key Idea

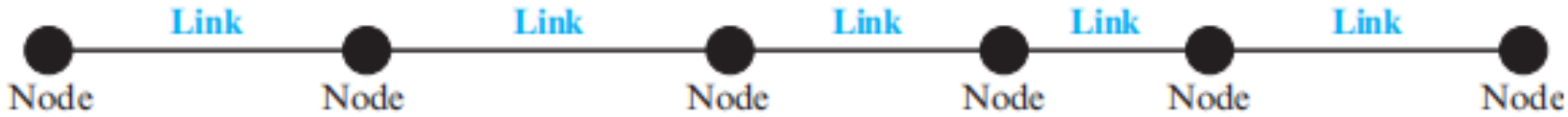
Network Layer = **Host-to-Host**

Data-Link Layer = **Node-to-Node / Hop-to-Hop**

Figure 5.2 *Nodes and Links*



a. A small part of the Internet



b. Nodes and links

Data-Link Layer: Types of Links & Sublayers

2. Two Types of Links

- **Point-to-Point:** dedicated link between exactly two devices; the entire link capacity is available to them.
- **Broadcast:** shared link/medium used by several devices; devices compete for the shared capacity.
- **Example:** traditional home telephone $\rightarrow$ point-to-point; cellular communication $\rightarrow$ broadcast/shared wireless medium.

3. Two Data-Link Sublayers

- **Data Link Control (DLC):** handles functions common to both point-to-point and broadcast links.
- **Media Access Control (MAC):** handles functions specific to broadcast/shared links.

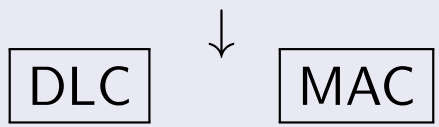
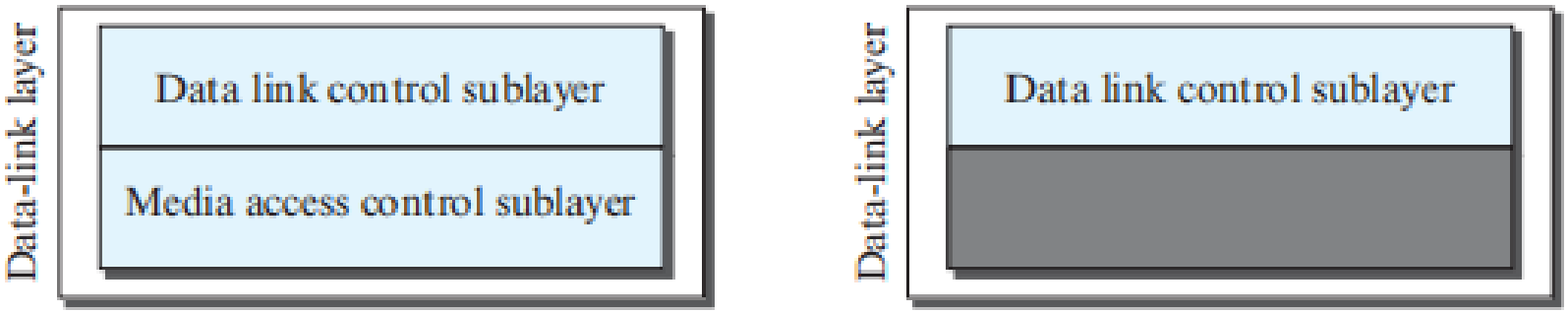


Figure 5.3 *Dividing the data-link layer into two sublayers*



a. Data-link layer of a broadcast link

b. Data-link layer of a point-to-point link

5.2 Data Link Control (DLC)

Purpose

Data Link Control (DLC) manages communication between **adjacent nodes** (node-to-node), whether the link is point-to-point or broadcast.

Main Functions

- **Framing:** organizes the bit stream into distinguishable frames.
- **Flow control:** regulates the amount of data sent before acknowledgment.
- **Error control:** detects damaged/lost frames and coordinates retransmission.
- **Error detection/correction:** identifies corrupted data and, in some protocols, corrects it.

Framing

- Physical layer transmits a continuous stream of bits.
- Data-link layer groups these bits into separate **frames**.
- Frames distinguish one data unit from another and carry:
  - Sender address
  - Destination address
  - Data
  - Control/error information

5.2.1 Framing: Why and How?

Why Divide a Message into Frames?

- A complete message could be placed in one large frame, but flow and error control become inefficient.
- If one bit is corrupted, the entire large frame would require retransmission.
- Smaller frames restrict the effect of an error to only the affected frame.

Important

Variable-size frames require a mechanism to identify the **start and end** of every frame.

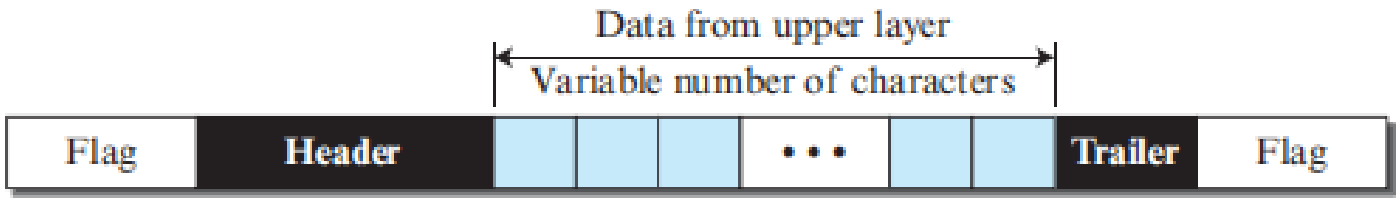
Two Approaches to Variable-Size Framing

1. **Character-oriented framing:** uses 8-bit characters and special byte flags.
2. **Bit-oriented framing:** treats data as a sequence of bits and uses a special bit pattern as the flag.

Frame Size

- **Fixed-size:** frame size itself identifies boundaries; e.g., ATM uses fixed-size cells.
- **Variable-size:** boundaries must be explicitly identified; widely used in LANs.

Figure 5.4 *A frame in a character-oriented protocol*



Character-Oriented Framing

Basic Structure

- Data consists of **8-bit characters**, such as ASCII.
- Header and trailer are also multiples of 8 bits.
- An **8-bit flag byte** is placed at the beginning and end of the frame.
- The flag identifies frame boundaries.

Limitation

8-bit character framing conflicts with modern **16-bit/32-bit character systems** such as Unicode; hence, bit-oriented protocols became more common.

Problem: Flag Appears in Data

If the same pattern as the flag occurs inside the data, the receiver may incorrectly interpret it as the end of the frame.

Byte / Character Stuffing

- Insert an **ESC (escape) byte** before a flag-like byte appearing in data.
- Receiver removes ESC and treats the following byte as ordinary data.
- If ESC itself occurs in the data, another ESC is inserted.
- Receiver removes the extra ESC during unstuffing.

Byte Stuffing: Example

Problem

Suppose the flag character appears naturally inside the data:

Data: A B FLAG C D

Without protection, the receiver may interpret **FLAG** as the end of the frame.

Solution: ESC

Insert an escape character before the flag:

A B ESC FLAG C D

The receiver removes ESC and treats FLAG as ordinary data.

If ESC Appears in Data

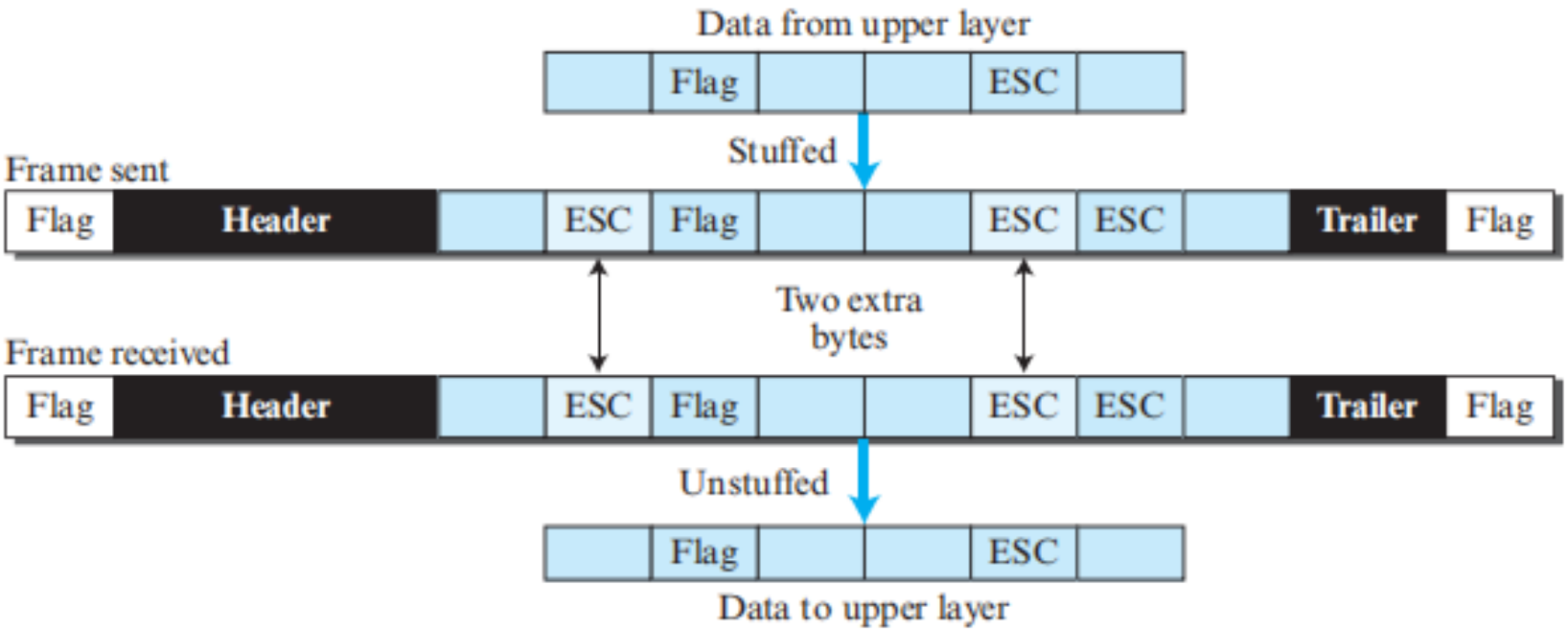
An ESC occurring naturally in the data must also be escaped:

ESC $\rightarrow$ ESC ESC

Therefore:

- **Flag in data** $\rightarrow$ insert ESC.
- **ESC in data** $\rightarrow$ insert another ESC.
- Receiver removes the additional ESC during unstuffing.

Figure 5.5 *Byte stuffing and unstuffing*



Bit-Oriented Framing

Basic Idea

- Data is treated as a sequence of bits and may represent text, graphics, audio, video, etc.
- A special **8-bit flag** identifies frame boundaries:

$$\boxed{01111110}$$

- The same flag pattern must not accidentally occur inside the data.

Example

$$01111110 \quad \rightarrow \quad 011111010$$

The stuffed pattern cannot be mistaken for the real flag.

Bit Stuffing

If the flag-like pattern occurs in data, the sender inserts one extra **0** after a sequence of **0 followed by five 1s**.

$$0\,11111 \quad \rightarrow \quad 0\,111110$$

The receiver removes this stuffed 0.

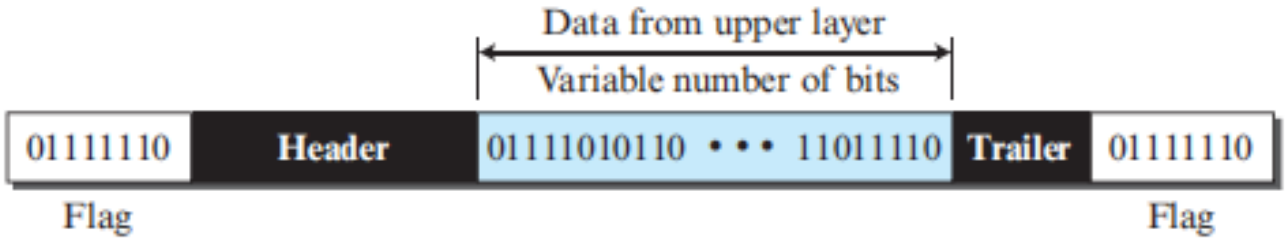
Important Rule

The extra 0 is inserted after every

$$\boxed{0\,11111}$$

regardless of the value of the next bit.

Figure 5.6 *A frame in a bit-oriented protocol*



Bit Stuffing: Sender and Receiver

At the Sender

1. Scan the data bit by bit.
2. After every sequence of **0 followed by five 1s**, insert an additional **0**.
3. Transmit the resulting stuffed frame.

Example

Original flag-like data:

$$01111110$$

After stuffing:

$$011111010$$

The resulting sequence is no longer interpreted as the flag.

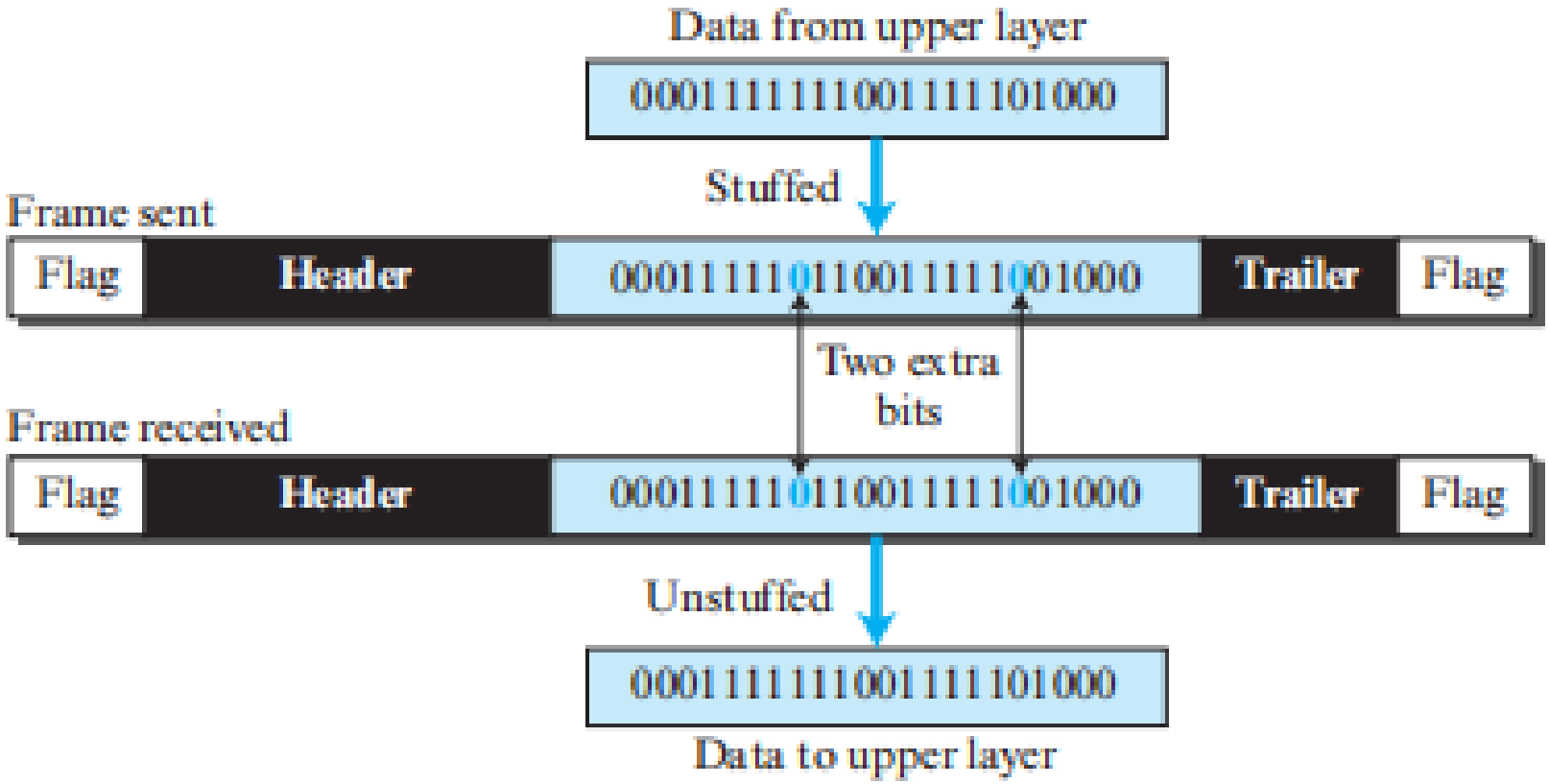
At the Receiver

1. Detect the flag:

$$\boxed{01111110}$$

2. Examine the received data.
3. Whenever a stuffed 0 follows five consecutive 1s, remove it.
4. Recover the original data.

Figure 5.7 *Bit stuffing and unstuffing*



Framing: Quick Comparison

Character-Oriented

- Data handled as **8-bit characters**.
- Uses special **byte flag**.
- Flag appearing in data $\rightarrow$ **byte stuffing**.
- ESC appearing in data $\rightarrow$ additional ESC.
- Limitation with modern 16/32-bit character systems.

Bit-Oriented

- Data handled as a **bit sequence**.
- Common flag:

$$01111110$$

- Flag-like sequence in data $\rightarrow$ **bit stuffing**.
- Insert 0 after a 0 followed by five 1s.
- Receiver removes the stuffed 0.

Exam Point

Byte stuffing adds an ESC byte; **Bit stuffing** adds one 0 bit.

5.2.2 Flow and Error Control

Basic Idea

The data-link control sublayer is responsible for both **flow control** and **error control** at the data-link layer.

Flow Control

Flow control coordinates the amount of data that can be sent before receiving an acknowledgment.
The data-link layer follows the same basic principle used for flow control at the transport layer.

Important Difference

Transport Layer:

- End-to-end
- Host-to-host

Data-Link Layer:

- Node-to-node
- Across a single link

Flow control at the data-link layer is concerned with communication between **two adjacent nodes**

Error Control at the Data-Link Layer

What is Error Control?

Error control includes both **error detection** and **error correction**.

- Allows the receiver to inform the sender about frames that are **lost or damaged**.
- Coordinates the **retransmission** of damaged or lost frames.
- In the data-link layer, error control primarily refers to:
  - Error detection
  - Retransmission
- A common implementation is:

Error detected $\rightarrow$ **Corrupted frame discarded** $\rightarrow$ **Frame retransmitted**

Key Difference

Transport-layer error control is **end-to-end**, whereas data-link-layer error control is **node-to-node**.

Flow and Error Control: Transport vs Data Link

	Transport Layer	Data-Link Layer
Scope	End-to-end	Node-to-node
Communication	Host-to-host	Adjacent nodes
Flow control	Amount of data before ACK	Amount of data before ACK
Error control	End-to-end recovery	Recovery across one link
Retransmission	End-to-end	Across the current link

Analogy

Transport Layer = entire journey
Data-Link Layer = one road segment between two adjacent cities

5.2.3 Error Detection and Correction

Why Error Detection?

At the data-link layer, if a frame is corrupted between two nodes, it needs to be handled before it continues its journey to other nodes.

- Most link-layer protocols simply **discard** the corrupted frame.
- Upper-layer protocols may then handle **retransmission**.
- Some wireless protocols attempt to **correct corrupted frames**.

Main Idea

Detect the error $\rightarrow$ Discard or correct the frame $\rightarrow$ Retransmit when necessary

Types of Errors

Why do errors occur?

Whenever bits flow from one point to another, they may undergo unpredictable changes because of **interference**. Interference can change the shape of the signal.

Single-Bit Error

Only **one bit** of a data unit changes.

$$0 \rightarrow 1 \qquad \text{or} \qquad 1 \rightarrow 0$$

Example:

$$10110101$$

becomes

$$10100101$$

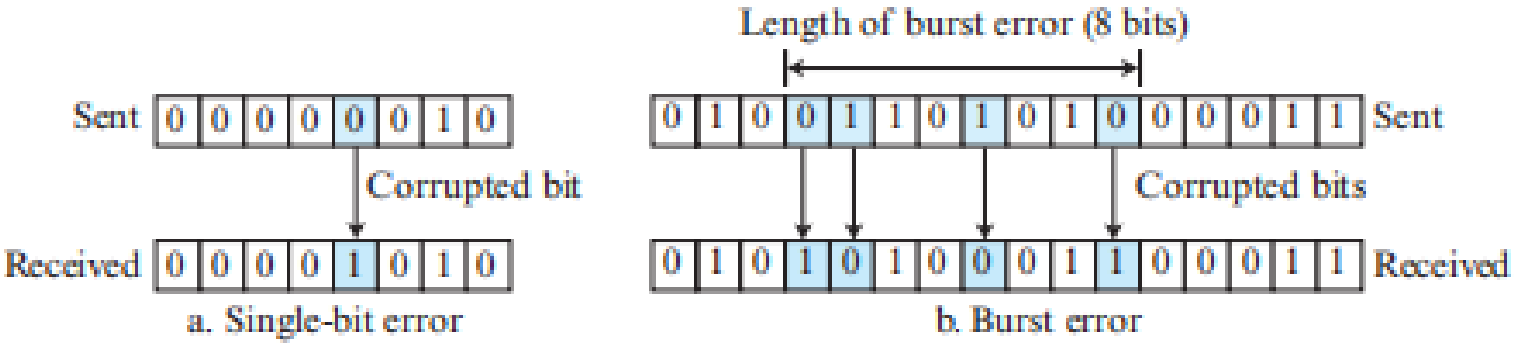
Burst Error

Two or more bits in a data unit are changed.
Example:

$$101101011001$$

contains multiple corrupted bits.

Figure 5.8 *Single-bit and burst error*



Burst Errors and Data Rate

The number of bits affected by a burst error depends on:

- Data rate
- Duration of the noise

Example

Suppose the duration of noise is:

$$\frac{1}{100} \text{ second}$$

Data Rate	Noise Duration	Bits Affected
1 kbps	$1/100$ sec	10 bits
1 Mbps	$1/100$ sec	10,000 bits

Observation

The same noise duration can affect vastly more bits at a higher data rate.

Redundancy

Central Concept

Redundancy

To detect or correct errors, we need to send **extra bits** along with the actual data.

$\longrightarrow$

Sender

- Adds redundant bits
- Sends data + redundancy

Receiver

- Uses redundant bits
- Detects/corrects errors
- Removes redundancy

Key Idea

Redundant bits create a relationship between the original data bits and the additional bits.

Detection versus Correction

Error Detection

We only need to determine:

Did an error occur?

Answer:

YES or NO

The exact number and location of corrupted bits are not necessarily required.

Error Correction

We need to determine:

- Number of corrupted bits
- Exact location of corrupted bits

Therefore correction is more difficult.

Example

For an 8-bit data unit:

- One error $\rightarrow$ 8 possible locations and Two errors $\rightarrow \binom{8}{2} = 28$ possible locations

Coding

Purpose of Coding

Redundancy is achieved using different **coding schemes**.

- Sender adds redundant bits.
- A relationship is created between:
  - Actual data bits
  - Redundant bits
- Receiver checks this relationship.
- Errors can then be detected.

Two Broad Categories

1. **Block coding**
2. **Convolution coding**

This Chapter

The discussion concentrates mainly on **block coding**.

Block Coding: Datawords and Codewords

In block coding:

$$\boxed{k \text{ data bits}} \quad + \quad \boxed{r \text{ redundant bits}} \quad = \quad \boxed{n \text{ codeword bits}}$$

where

$$n = k + r$$

Dataword

A block of k bits.
Number of possible datawords:

$$2^k$$

Codeword

A block of n bits.
Number of possible codewords:

$$2^n$$

Block Coding: Datawords and Codewords

Since

$$n > k$$

we have:

$$2^n > 2^k$$

Therefore:

$$\boxed{2^n - 2^k}$$

possible codewords are unused and are called **invalid/illegal codewords**.

Block Coding and Error Detection

An error can be detected if two conditions are satisfied:

1. The receiver has, or can determine, a list of **valid codewords**.
2. The original codeword changes into an **invalid codeword**.

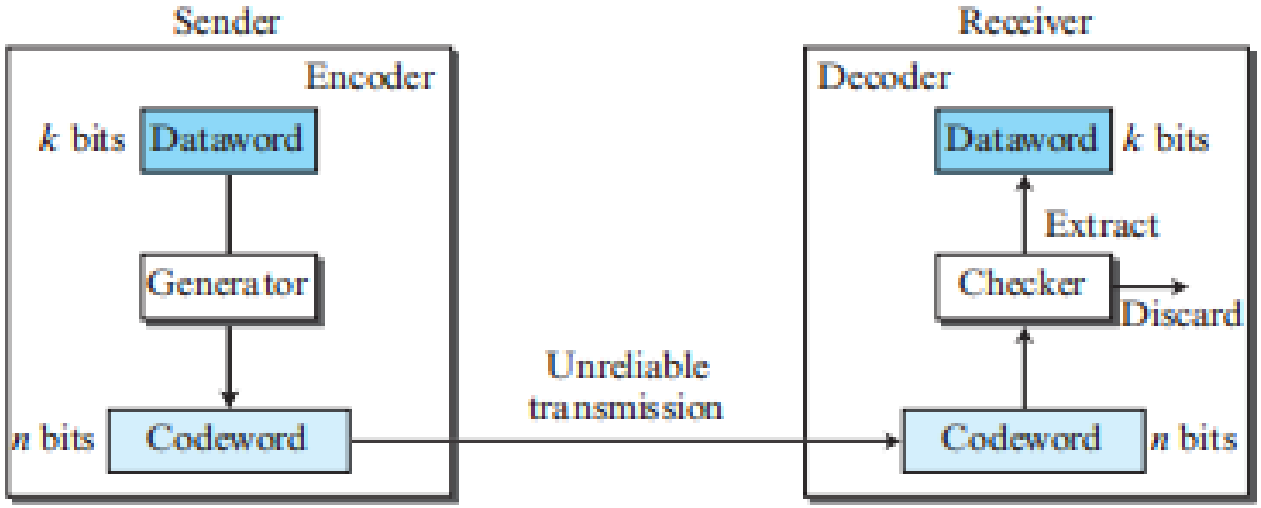
Receiver Decision

- Received word is valid $\rightarrow$ accept it.
- Received word is invalid $\rightarrow$ discard it.

Limitation

If corruption changes one valid codeword into another valid codeword, the error remains **undetected**.

Figure 5.9 *Process of error detection in block coding*



Example 5.1: Block Coding

Let:

$$k = 2, \qquad n = 3$$

Dataword	Codeword	Dataword	Codeword
00	000	10	101
01	011	11	110

Assume:

Dataword $01 \rightarrow$ Codeword 011

1. Receiver gets **011**: valid $\rightarrow$ extracts 01.
2. Receiver gets **111**: invalid $\rightarrow$ discarded.
3. Receiver gets **000**: valid $\rightarrow$ incorrectly extracts 00.

Conclusion

Two corrupted bits changed 011 into another valid codeword, so the error was **undetected**.

Hamming Distance

Definition

The Hamming distance between two words of the same size is the number of differences between their corresponding bits.

$$\boxed{d(x, y) = \text{number of differing bit positions}}$$

XOR Method

Hamming distance can be found by XORing the two words and counting the number of 1s.

Hamming Distance

Example

If:

$$x = 00000$$

and

$$y = 01101$$

then three positions differ.
Therefore:

$$\boxed{d(00000, 01101) = 3}$$

Example 5.2: Hamming Distance

Example 1

$$d(000, 011)$$

XOR:

$$000 \oplus 011 = 011$$

Number of 1s $= 2$. Therefore:

$$\boxed{d(000, 011) = 2}$$

Example 2

$$d(10101, 11110)$$

XOR:

$$10101 \oplus 11110 = 01011$$

Number of 1s $= 3$. Therefore:

$$\boxed{d(10101, 11110) = 3}$$

Minimum Hamming Distance

Definition

The minimum Hamming distance, $d_{\min}$, is the smallest Hamming distance between any pair of valid codewords.

To guarantee detection of up to s errors:

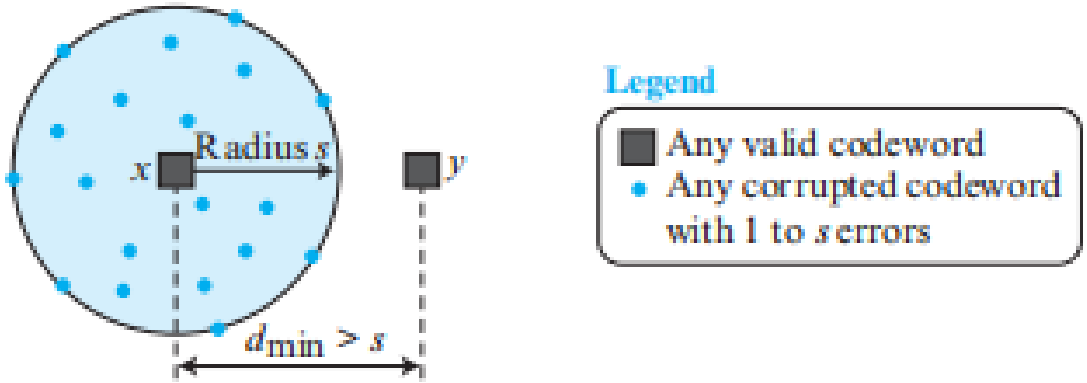
$$\boxed{d_{\min} = s + 1}$$

Therefore:

$$\boxed{s = d_{\min} - 1}$$

$d_{\min}$	Guaranteed Error Detection
2	1 error
3	2 errors
4	3 errors
5	4 errors

Figure 5.10 *Geometric concept explaining d_{min} in error detection*



Examples 5.3 and 5.4

Important

Only s or fewer errors are guaranteed to be detected.

Example 5.3

The first code has:

$$d_{\min} = 2$$

Therefore:

$$s = d_{\min} - 1 = 1$$

It guarantees detection of only a **single error**.
Two errors may transform one valid codeword into another valid codeword.

Example 5.4

A code has:

$$d_{\min} = 4$$

Therefore:

$$s = 4 - 1 = 3$$

The code guarantees detection of up to **three errors**.

Linear Block Codes

Definition

A linear block code is a code in which the **exclusive OR (XOR)** of any two valid codewords produces another valid codeword.

$$\text{Valid Codeword} \oplus \text{Valid Codeword} = \text{Valid Codeword}$$

Example 5.5

The code in Table 5.1 is a linear block code because XORing any codeword with another codeword produces a valid codeword.

Minimum Distance

For a linear block code:

$$d_{\min} = \text{minimum number of 1s in a nonzero valid codeword}$$

Example 5.6: Minimum Distance

Consider the first code:

Codewords
000
011
101
110

The nonzero codewords are:

$$011, \qquad 101, \qquad 110$$

Each has exactly two 1s.
Therefore:

$$\boxed{d_{\min} = 2}$$

Hence this code guarantees detection of:

$$\boxed{d_{\min} - 1 = 1 \text{ error}}$$

Parity-Check Code

Definition

A parity-check code is a familiar error-detecting code and is a **linear block code**.

For a k-bit dataword:

$$\boxed{n = k + 1}$$

The additional bit is called the **parity bit**.

For even parity, the parity bit is selected so that the total number of 1s in the codeword is even.

$$\boxed{d_{\text{min}} = 2}$$

Therefore:

$$\boxed{\text{Single-bit error detection}}$$

Simple Parity-Check Code $C(5, 4)$

Dataword	Codeword	Dataword	Codeword
0000	00000	1000	10001
0001	00011	1001	10010
0010	00101	1010	10100
0011	00110	1011	10111
0100	01001	1100	11000
0101	01010	1101	11011
0110	01100	1110	11101
0111	01111	1111	11110

The parity bit is:

$$\boxed{r_0 = a_3 \oplus a_2 \oplus a_1 \oplus a_0}$$

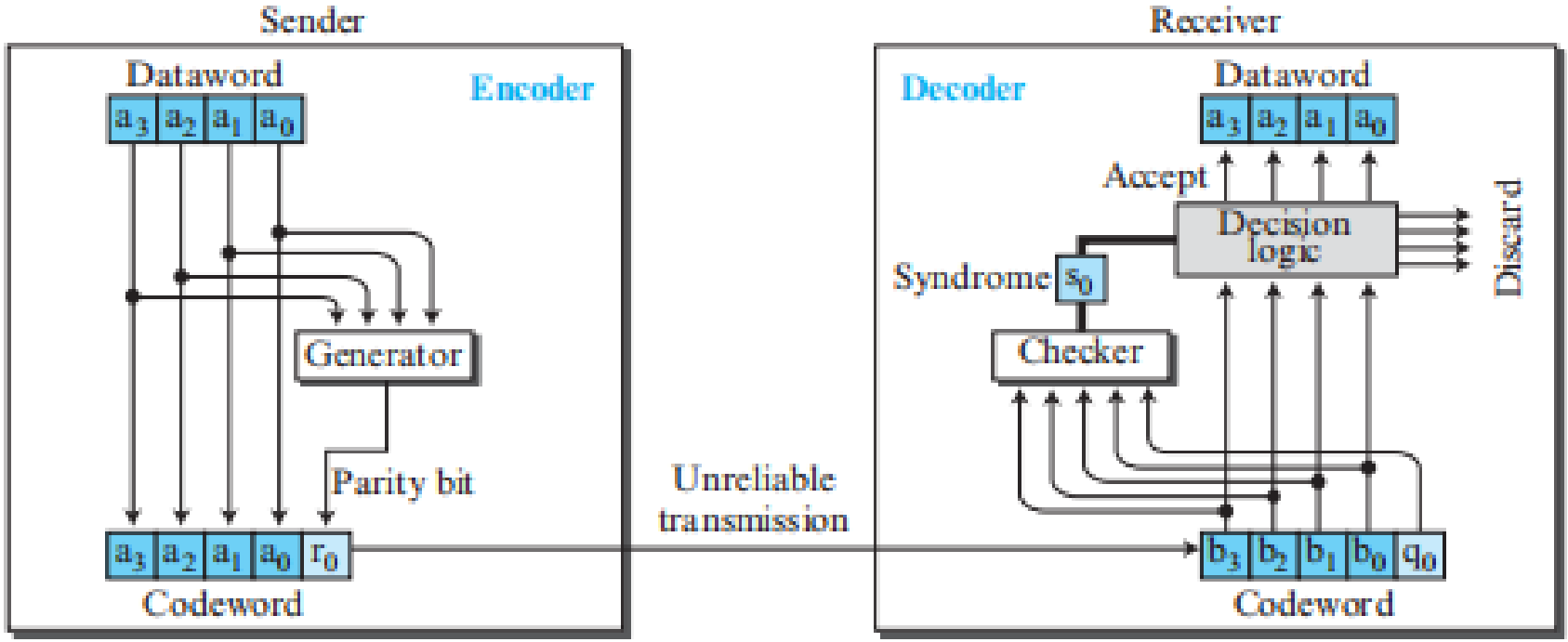
or equivalently,

$$r_0 = a_3 + a_2 + a_1 + a_0 \pmod 2$$

Rule

If the number of 1s in the dataword is even, $r_0 = 0$.
If the number of 1s is odd, $r_0 = 1$.

Figure 5.11 *Encoder and decoder for simple parity-check code*



Parity-Check Encoder and Decoder

Encoder

1. Take 4-bit dataword:

$$a_3 a_2 a_1 a_0$$

2. Generate parity bit:

$$r_0 = a_3 \oplus a_2 \oplus a_1 \oplus a_0$$

3. Create 5-bit codeword.

Decoder

1. Receive 5-bit word.
2. Perform modulo-2 addition over all 5 bits.
3. Result is the **syndrome**.

Syndrome

$$\boxed{S = 0 \Rightarrow \text{No detectable error}}$$

$$\boxed{S = 1 \Rightarrow \text{Error detected}}$$

Example 5.7: Parity-Check Scenarios

Assume the sender sends:

$$\text{Dataword} = 1011$$

The generated codeword is:

$$\boxed{10111}$$

1. **No error:** received = 10111, syndrome = 0. Dataword 1011 is created.
2. **One error in a_1:** received = 10011, syndrome = 1. No dataword is created.
3. **One error in r_0:** received = 10110, syndrome = 1. No dataword is created.
4. **Two errors:** received = 00110, syndrome = 0. Dataword 0011 is incorrectly created.
5. **Three errors:** received = 01011, syndrome = 1. Dataword is not created.

Conclusion

Simple parity can detect a single error and, in general, any **odd number of errors**, but it cannot detect an **even number of errors**.

Cyclic Codes

Definition

Cyclic codes are special linear block codes with one additional property.

If a codeword is cyclically shifted, the result is also a codeword.

Example:

$$1011000$$

Cyclic left shift:

$$\boxed{0110001}$$

Key Property

A cyclic shift of a valid codeword produces another valid codeword.

Cyclic Redundancy Check (CRC)

CRC

Cyclic Redundancy Check is a subset of cyclic codes widely used in networks such as LANs and WANs.

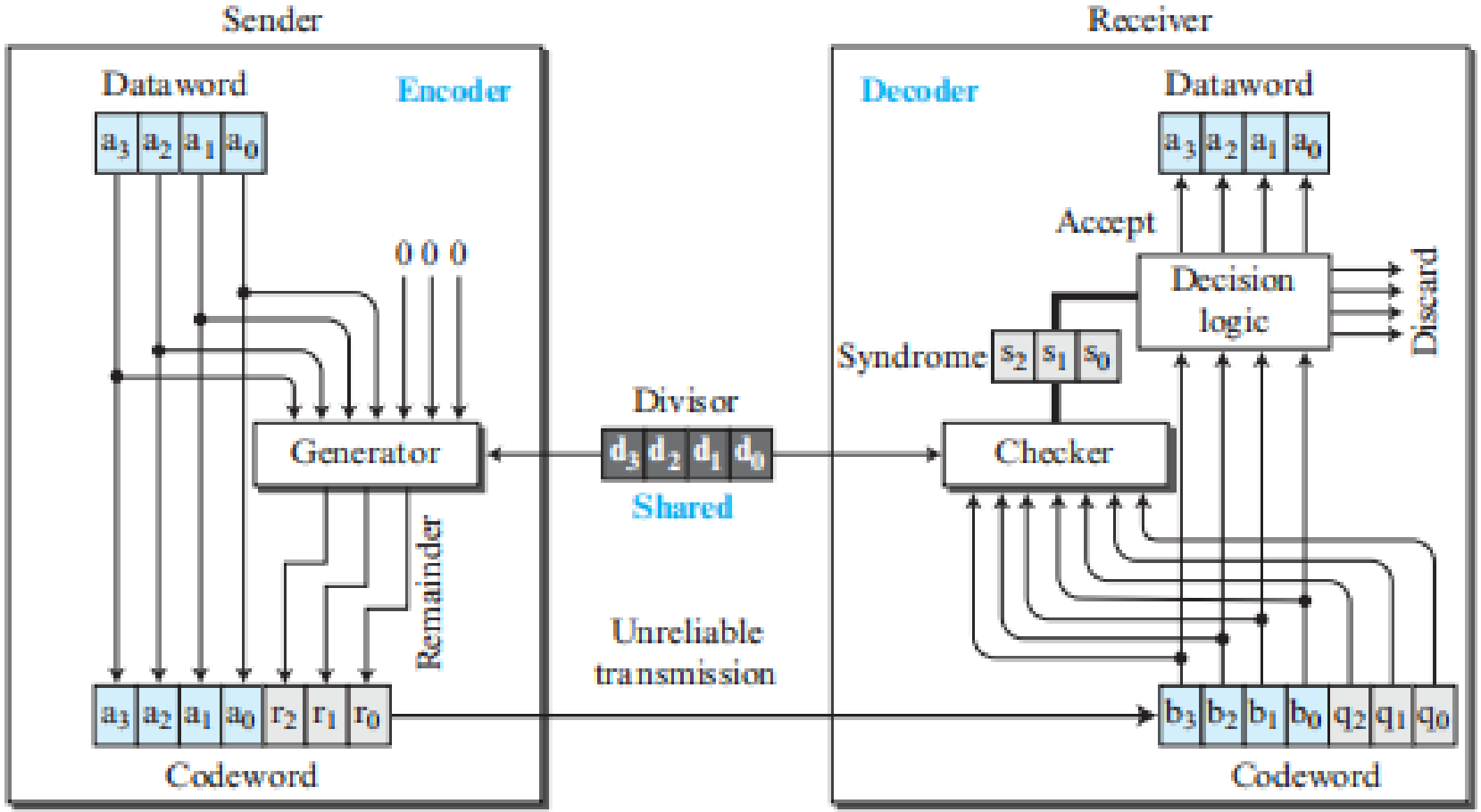
Encoder

1. Dataword has k bits.
2. Append $n - k$ zeros.
3. Divide by predefined divisor.
4. Use modulo-2 division.
5. Remainder becomes check bits.
6. Append remainder to dataword.

Decoder

1. Receive codeword.
2. Divide by same divisor.
3. Calculate syndrome.
4. Syndrome $= 0$: accept.
5. Syndrome $\neq 0$: discard.

Figure 5.12 *CRC encoder and decoder*



CRC: Encoder Process

For the example:

$$k = 4, \qquad n = 7$$

Therefore:

$$n - k = 3$$

The dataword is augmented with three zeros:

$$\boxed{\text{Dataword} + 000}$$

The generator/divisor has:

$$n - k + 1 = 4 \text{ bits}$$

The augmented dataword is divided by the divisor using **modulo-2 division**.

Important

The quotient is discarded.

Modulo-2 Division

Modulo-2 binary division is similar to ordinary binary division, except:

$$\boxed{\text{Addition} = \text{Subtraction} = \text{XOR}}$$

At each step:

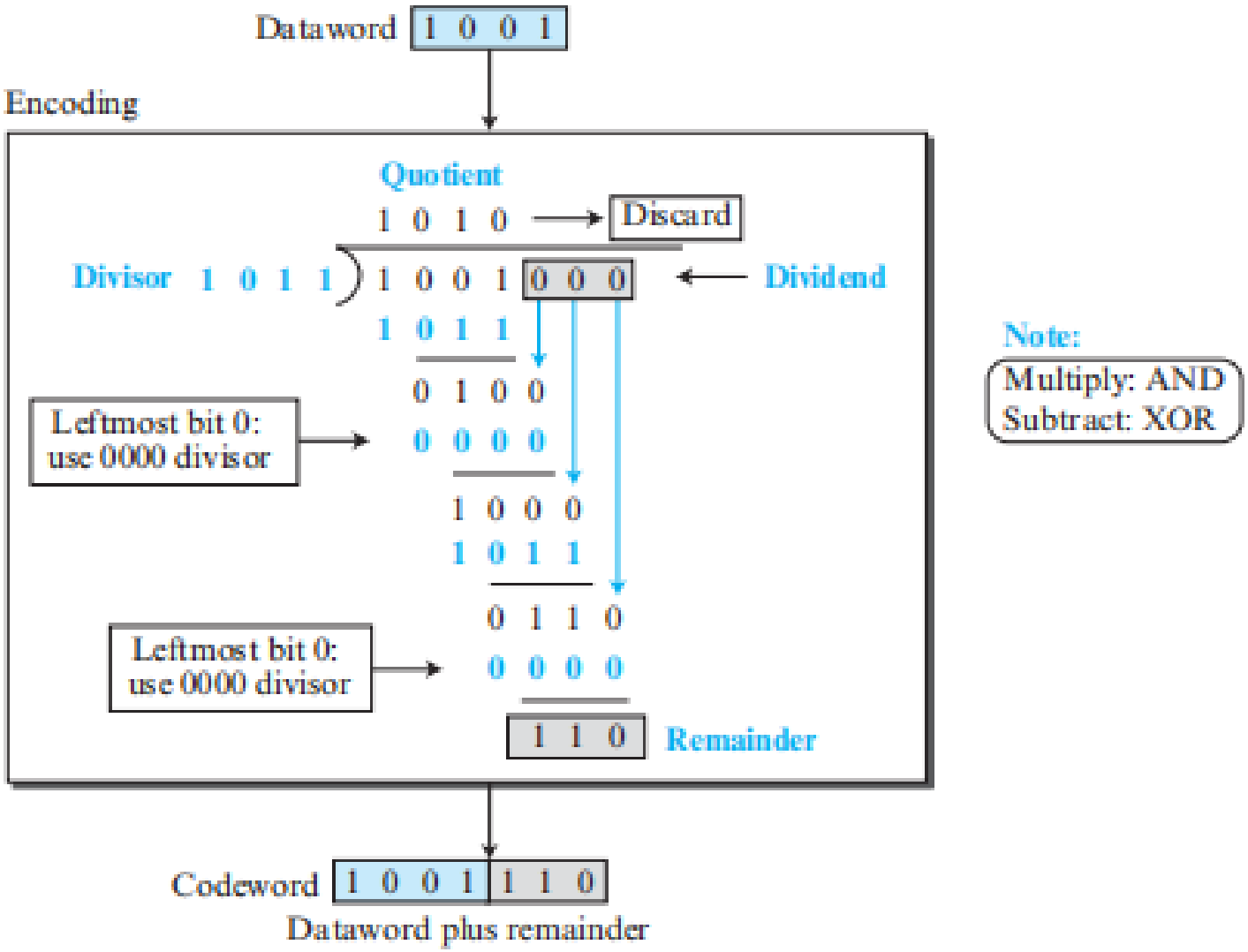
- XOR the divisor with the current 4-bit portion.
- The result is a 3-bit remainder.
- Pull down the next bit.
- Continue the process.

Important Rule

If the leftmost bit of the current dividend is 0, an all-zero divisor is used for that step.

When all bits have been processed, the final 3-bit remainder forms:

Figure 5.13 *Division in CRC encoder*



CRC Decoder

The decoder performs the same division process as the encoder.

Decision

The remainder is called the **syndrome**.

$$\boxed{\text{Syndrome} = 000 \Rightarrow \text{No error detected}}$$

$$\boxed{\text{Syndrome} \neq 000 \Rightarrow \text{Error detected}}$$

No Error

Syndrome:

$$000$$

The dataword is separated and accepted.

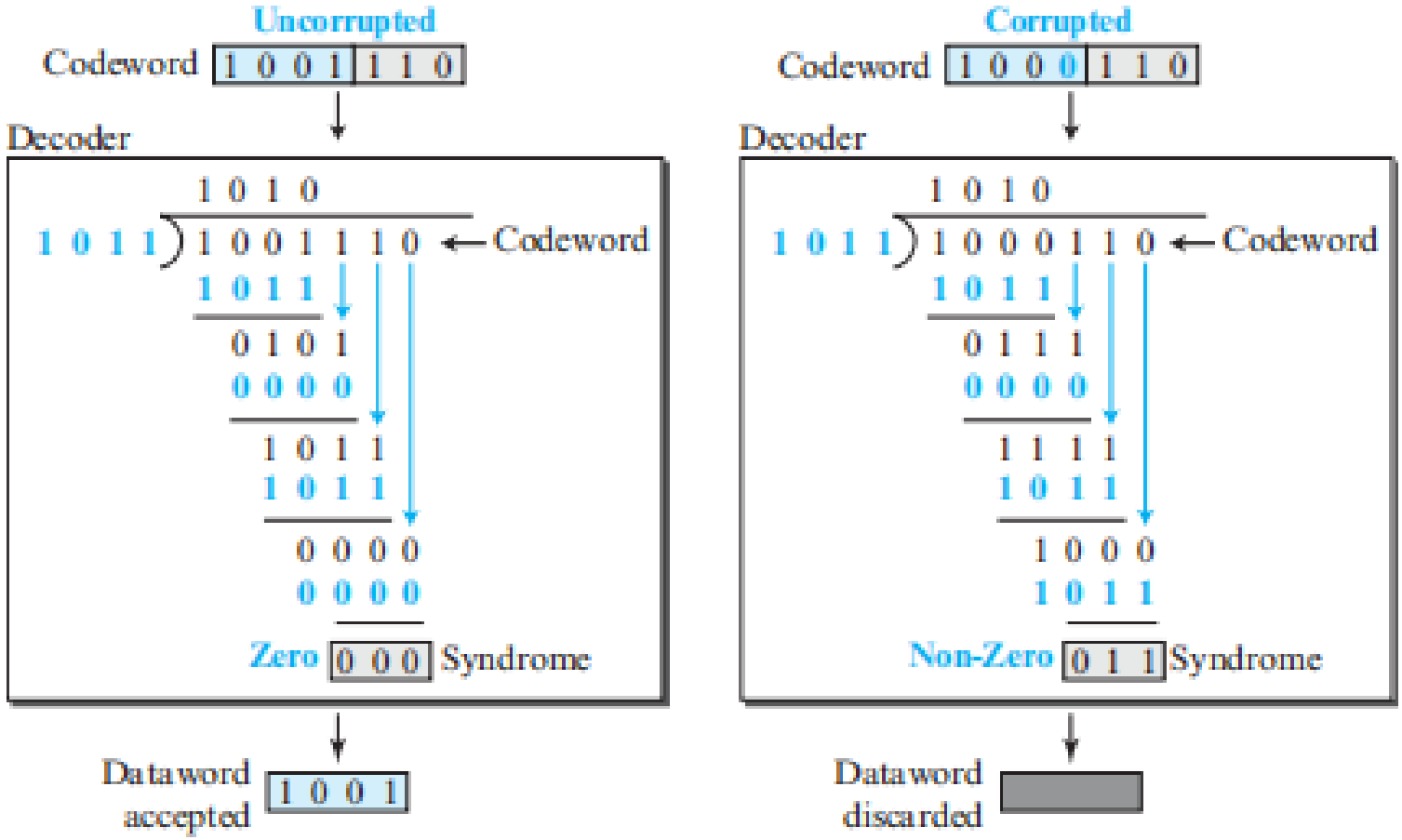
Error

Example syndrome:

$$011$$

The received data is discarded.

Figure 5.14 *Division in the CRC decoder for two cases*



CRC Divisor

How is the divisor selected?

The divisor depends on the error-detection requirements expected from the code.

Requirements for a Generator

A valid generator/divisor must have:

1. At least two bits.
2. Both the leftmost and rightmost bits must be 1.

CRC Naming

The number in CRC-n refers to the degree of the polynomial.
Therefore the divisor contains:

$$\boxed{n+1 \text{ bits}}$$

Example:

Standard CRC Polynomials

Name	Binary	Application
CRC-8	100000111	ATM header
CRC-10	11000110101	ATM AAL
CRC-16	10001000000100001	HDLC
CRC-32	100000100110000010001110110110111	LANs

Remember

CRC-32 has a polynomial degree of 32, so its divisor contains 33 bits.

CRC Performance

Single Errors

All qualified generators can detect **any single-bit error**.

Odd Number of Errors

Qualified generators can detect every odd number of errors if the generator is evenly divisible by:

$$(11)_2$$

using modulo-2 arithmetic.

Burst Errors

Let:

$$L = \text{length of burst error}$$

$$r = \text{length of remainder}$$

Then:

- $L \leq r$: all burst errors are detected.
- $L = r + 1$: detected with probability

$$1 - (0.5)^{r-1}$$

- $L > r + 1$: detected with probability

$$1 - (0.5)^{r}$$

Advantages of Cyclic Codes

Why are CRCs widely used?

- Easy to implement in hardware.
- Easy to implement in software.
- Very fast when implemented in hardware.
- Suitable for many computer networks.

Hardware Implementation

CRC division can be implemented using a **shift register** inside network hardware.

Checksum

Definition

Checksum is an error-detecting technique that can be applied to a message of any length.

- At the source, the message is divided into m-bit units.
- The generator creates an additional m-bit unit called the **checksum**.
- The checksum is transmitted with the message.
- The receiver calculates a new checksum.
- If the result is all 0s, the message is accepted.
- Otherwise, the message is discarded.

Internet

Traditionally, the Internet has used a **16-bit checksum**.

Checksum: Basic Concept

Suppose the message contains five 4-bit numbers:

$$(7, 11, 12, 0, 6)$$

The ordinary sum is:

$$7 + 11 + 12 + 0 + 6 = 36$$

Therefore the sender could send:

$$\boxed{(7, 11, 12, 0, 6, 36)}$$

The receiver:

1. Adds the five original numbers.
2. Compares the result with 36.
3. If they match, the receiver assumes no error.

Problem 36 cannot be represented using only 4 bits.

One's Complement Addition

One solution is to use **one's complement arithmetic**.
With m bits, unsigned numbers from:

$$0 \quad \text{to} \quad 2^m - 1$$

can be represented.
If a result produces more than m bits:

Extra leftmost bits are wrapped around and added to the rightmost m bits.

For the previous example:

$$36 = (100100)_2$$

Using 4-bit one's complement arithmetic:

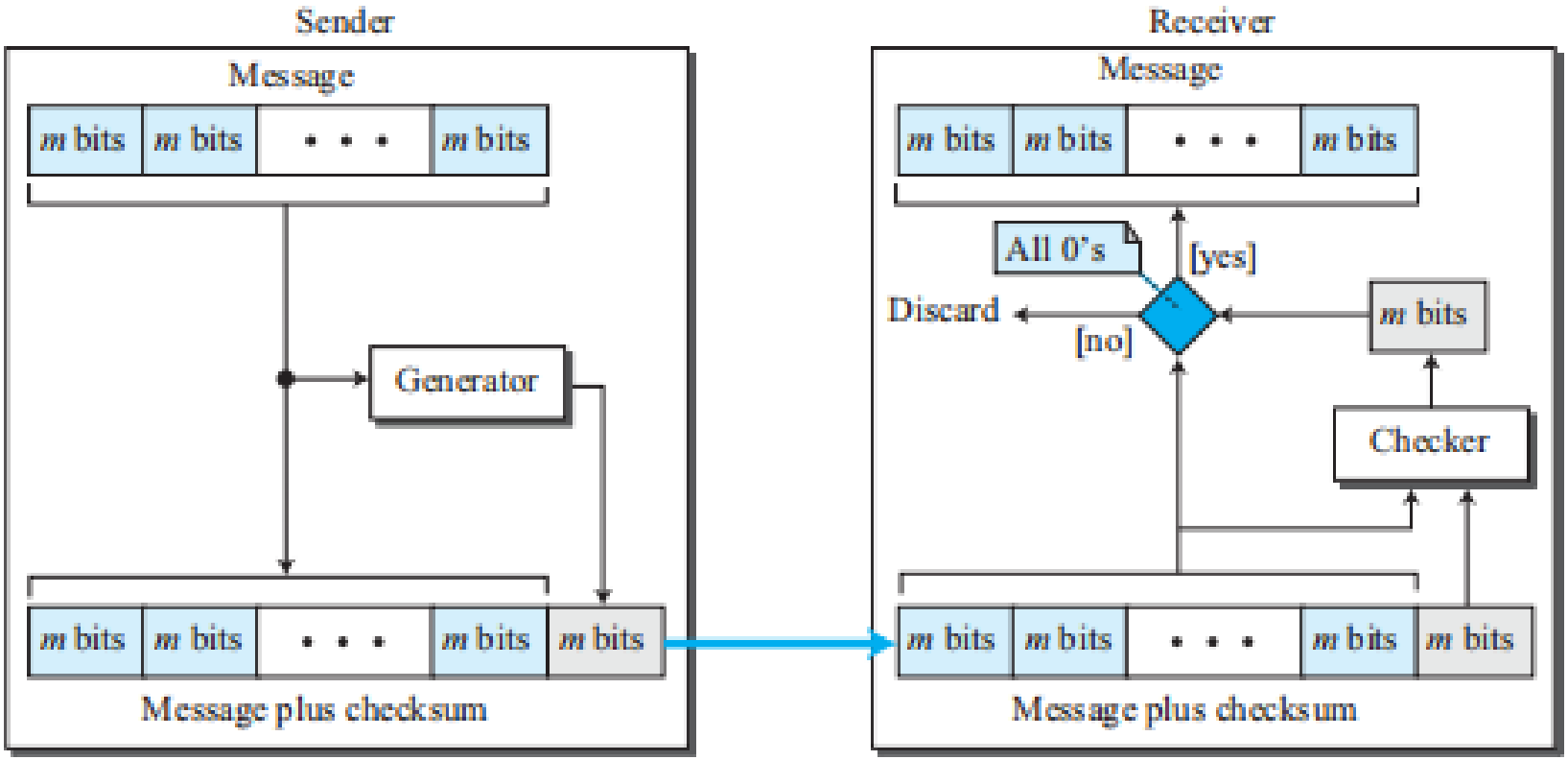
$$100100$$

Wrap the extra bit:

$$0100 + 10 = 0110$$

Therefore: $36 \rightarrow 6$

Figure 5.15 *Checksum*



Checksum Using One's Complement

Instead of sending the sum itself, the sender sends its **complement**.
Example:

$$6 = (0110)_2$$

Complement:

$$\boxed{1001}$$

Therefore:

$$\boxed{6 \rightarrow 9}$$

The sender transmits:

$$\boxed{(7, 11, 12, 0, 6, 9)}$$

At the receiver, all numbers are added using one's complement arithmetic. If the final result is:1111
this is the negative zero. Complementing it gives:0000 Therefore the message is accepted.

Example 5.10: Checksum

The sender adds:(7,11,12,0,6) using one's complement arithmetic. The sum is:6 Complement: 15-6=9 Therefore the checksum is:9 The sender transmits:(7,11,12,0,6,9) If there is no corruption, the receiver obtains:

$$15 = (1111)_2$$

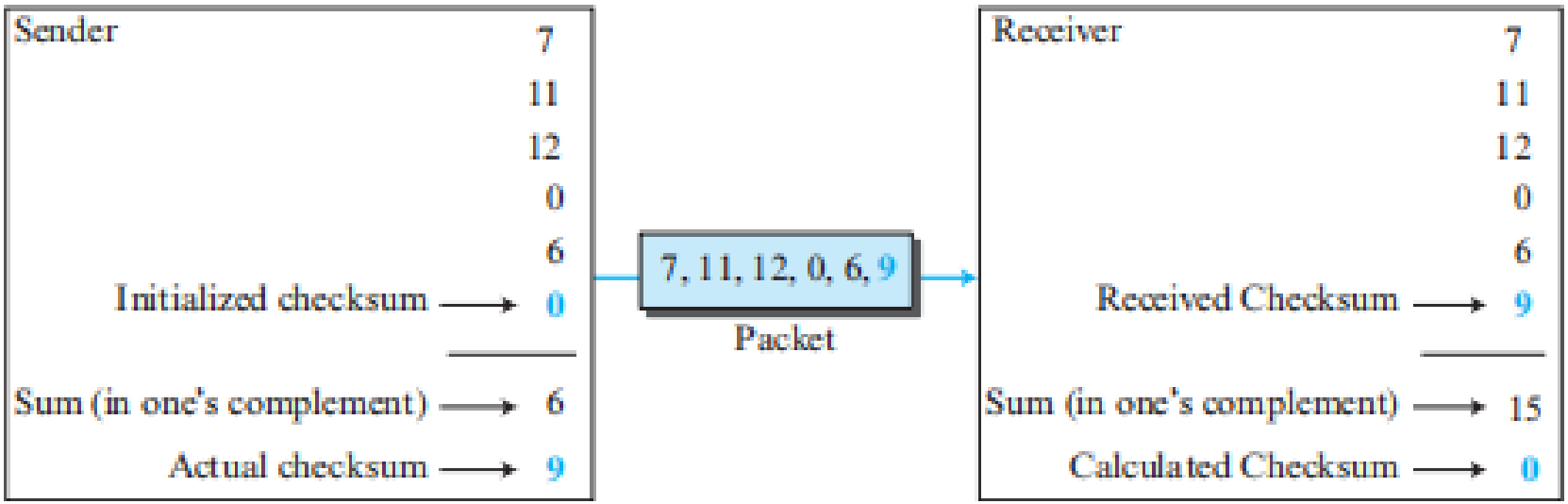
Complementing:

$$1111 \rightarrow 0000$$

Conclusion

A final result of zero indicates that the data has not been corrupted according to the checksum test.

Figure 5.16 *Example 5.10*



Internet Checksum: Procedure

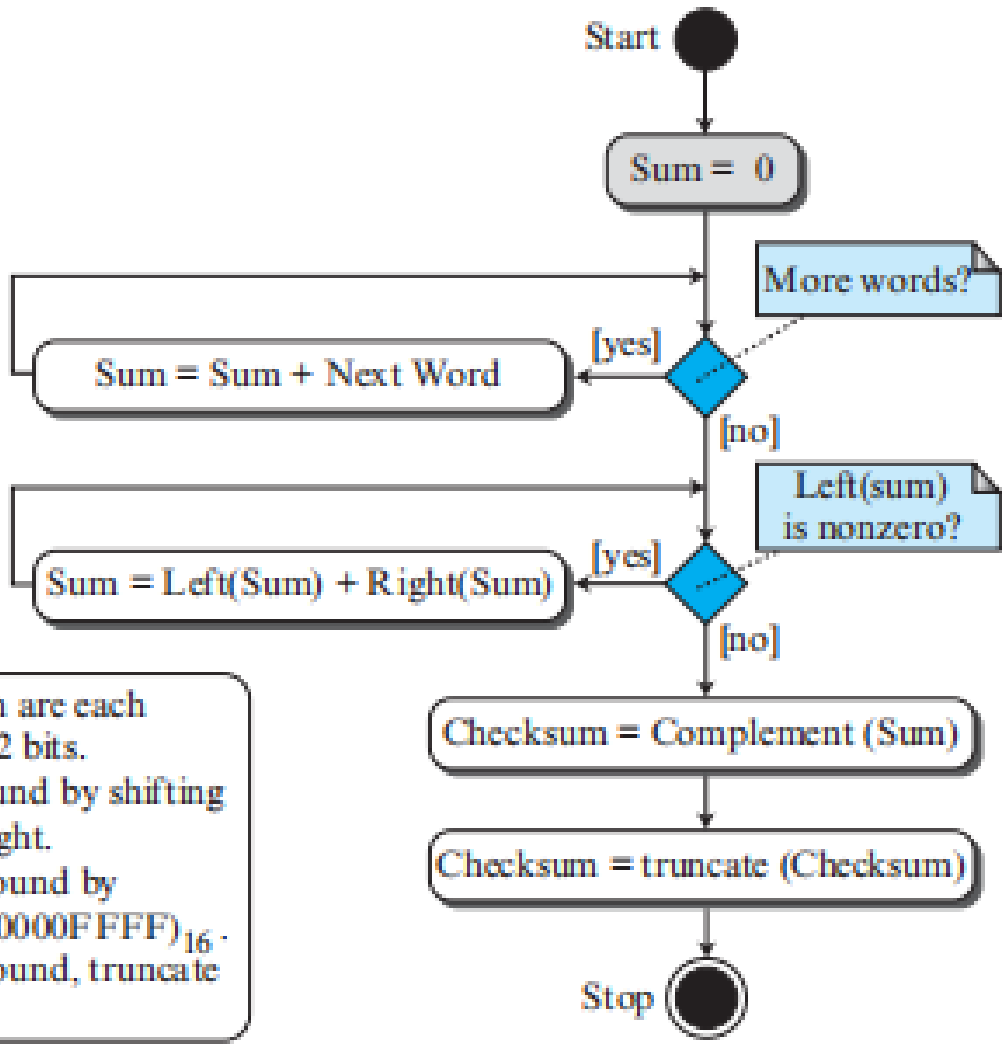
Table 5.5 *Procedure to calculate the traditional checksum*

Sender	*Receiver*
1. The message is divided into 16-bit words.	1. The message and the checksum is received.
2. The value of the checksum word is initially set to zero.	2. The message is divided into 16-bit words.
3. All words including the checksum are added using one's complement addition.	3. All words are added using one's complement addition.
4. The sum is complemented and becomes the checksum.	4. The sum is complemented and becomes the new checksum.
5. The checksum is sent with the data.	5. If the value of the checksum is 0, the message is accepted; otherwise, it is rejected.

Internet Checksum

The traditional Internet checksum uses **16-bit words**.

Figure 5.17 *Algorithm to calculate a traditional checksum*



Notes:

a. Word and Checksum are each 16 bits, but Sum is 32 bits.
b. Left(Sum) can be found by shifting Sum 16 bits to the right.
c. Right(Sum) can be found by ANDing Sum with $(0000FFFF)_{16}$.
d. After Checksum is found, truncate it to 16 bits.

Checksum: Performance

The traditional checksum uses only a small number of bits:

$$\boxed{16 \text{ bits}}$$

even for messages containing thousands of bits.

However, it is not as strong as CRC.

Example of Undetected Error

If one word is incremented and another word is decremented by the same amount:

$$+\Delta \quad \text{and} \quad -\Delta$$

the total sum remains unchanged.
Therefore the checksum may fail to detect the error.

Trend

For new protocols, there is a tendency to replace traditional checksums with **CRC**.

Other Approaches to Checksum

Problem with Traditional Checksum

Traditional checksum is **not weighted**.
Therefore it treats every data item equally.

If two 16-bit items are transposed:

$$A, B \longrightarrow B, A$$

the traditional checksum may not detect the error.

Solutions

Two approaches mentioned are:

1. **Fletcher checksum**
2. **Adler checksum**

Main Idea

These approaches introduce weighting into the checksum calculation.

Fletcher Checksum

Fletcher checksum was designed to give different weights to data items according to their positions.

Two versions are commonly described:

8-bit Fletcher

- Works on 8-bit data items.
- Produces a 16-bit checksum.
- Calculation modulo:

$$256 = 2^8$$

- Uses two accumulators:

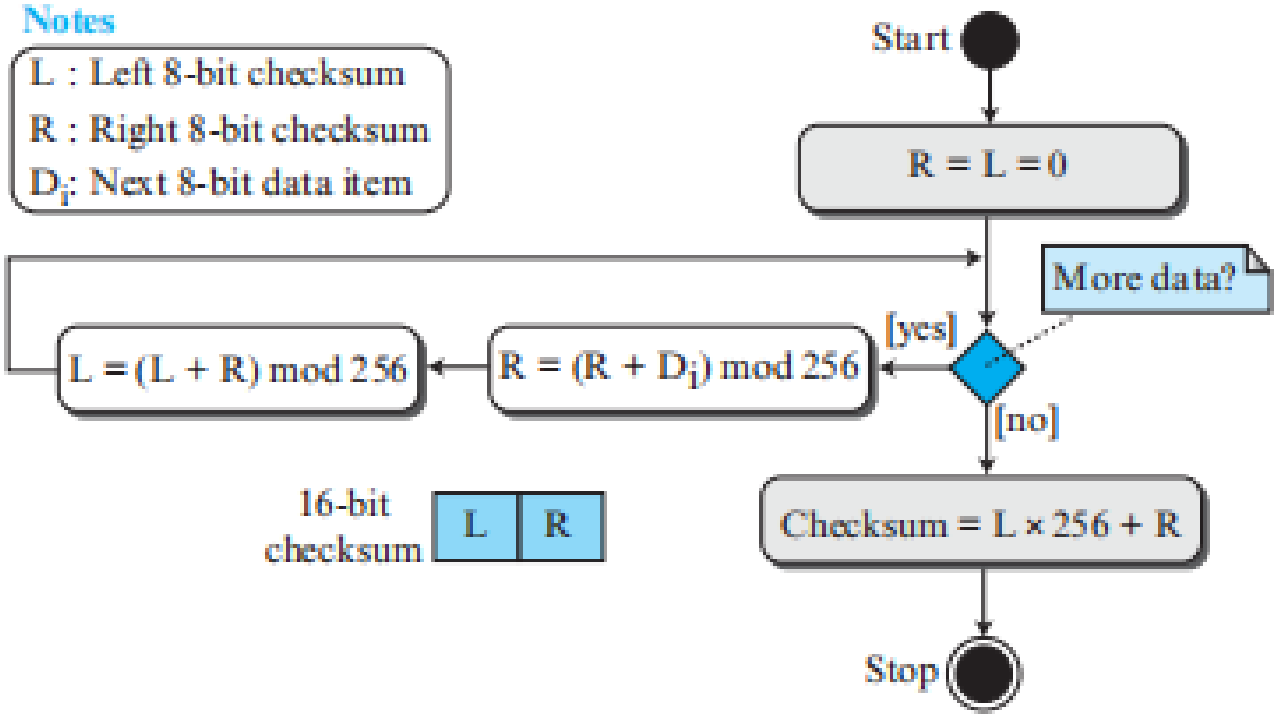
$$L,\ R$$

16-bit Fletcher

- Works on 16-bit data items.
- Produces a 32-bit checksum.
- Calculation modulo:

$$65,536$$

Figure 5.18 *Algorithm to calculate an 8-bit Fletcher checksum*



Adler Checksum

Adler checksum is a:

$$\boxed{32\text{-bit checksum}}$$

It is similar to the 16-bit Fletcher checksum, but has three important differences.

1. Calculation is performed on **single bytes**, rather than 2 bytes at a time.
2. The modulus is the prime number:

   $$65,521$$

   instead of 65,536.
3. The accumulator L is initialized to:

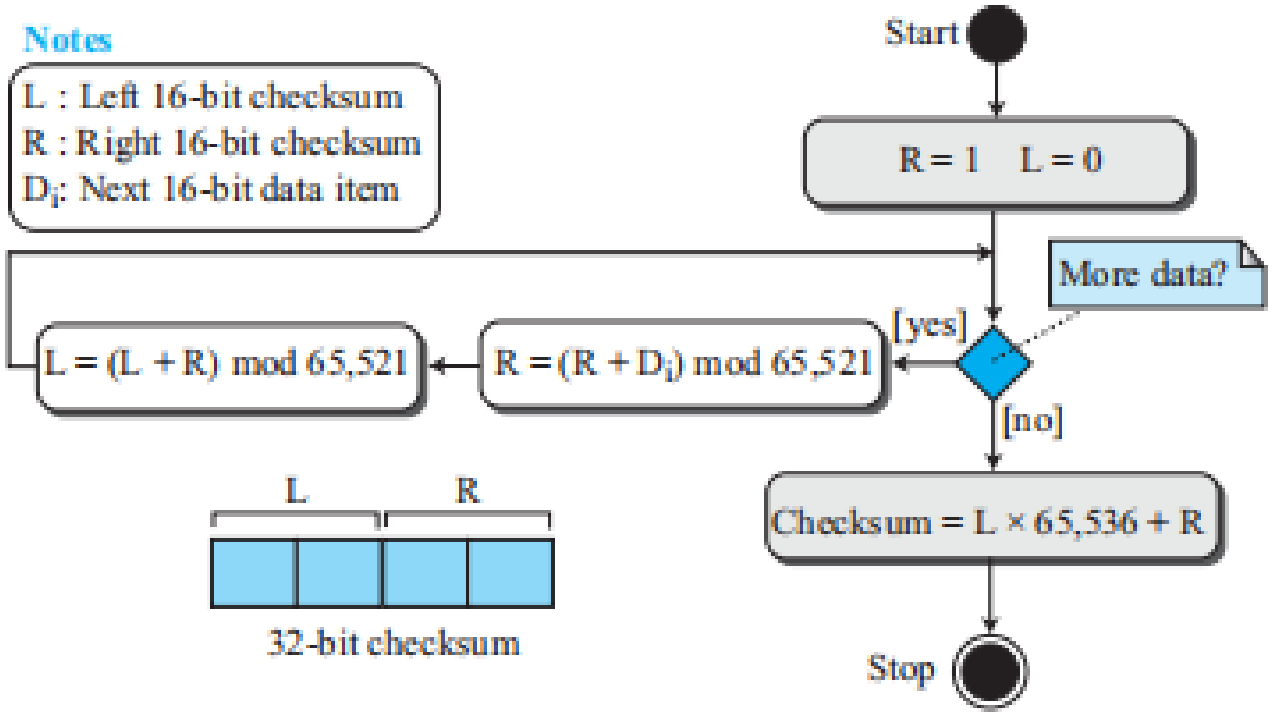
   $$L = 1$$

   instead of 0.

Why Prime Modulus?

A prime modulus has better error-detecting capability for some combinations of data.

Figure 5.19 *Algorithm to calculate an Adler checksum*



5.2.3 Error Detection: Big Picture

Technique	Main Idea	Key Property
Block Coding	Valid/invalid codewords	Detect invalid codes
Hamming Distance	Count bit differences	Determines detection capab
Parity Check	Add one parity bit	Detects single/odd errors
Cyclic Codes	Shifted codewords remain valid	Linear + cyclic
CRC	Polynomial/modulo-2 division	Strong error detection
Checksum	One's complement arithmetic	Simple, but weaker than CR
Fletcher	Weighted checksum	Better than traditional chec
Adler	Weighted checksum	32-bit checksum

Exam Formula

$$\boxed{d_{\min} = s + 1}$$

where s is the maximum number of errors guaranteed to be detected.

Data-Link Error Detection: Key Takeaways

- Errors occur because of interference during transmission.
- Errors can be:
  - Single-bit errors
  - Burst errors
- **Redundancy** is the basic idea behind error detection and correction.
- Block coding uses:

$$n = k + r$$

- Hamming distance determines error-detection capability.
- To guarantee detection of s errors:

$$d_{\min} = s + 1$$

- Parity check has:

$$d_{\min} = 2$$

- CRC is stronger and widely used in networks.
- Checksum is simple but generally weaker than CRC.

5.2.4 Two DLC Protocols

HDLC

- High-Level Data Link Control
- Bit-oriented DLC protocol
- Point-to-point and multipoint links
- Implements Stop-and-Wait
- Supports framing, flow/error control and error detection

PPP

- Point-to-Point Protocol
- Derived from HDLC concepts
- Used on point-to-point links

HDLC Transfer Modes: NRM and ABM

Normal Response Mode (NRM)

- **Unbalanced**: one Primary + one/more Secondary stations
- Primary sends commands and controls communication
- Secondary stations respond to the Primary
- Supports point-to-point and multipoint links

Insert Textbook Figure 5.20: NRM configuration

Asynchronous Balanced Mode (ABM)

- **Balanced**: both stations are peers
- Each can act as Primary or Secondary
- Used on point-to-point links
- Common mode used today

Insert Textbook Figure 5.21: ABM balanced configuration

Figure 5.20 *Normal response mode*

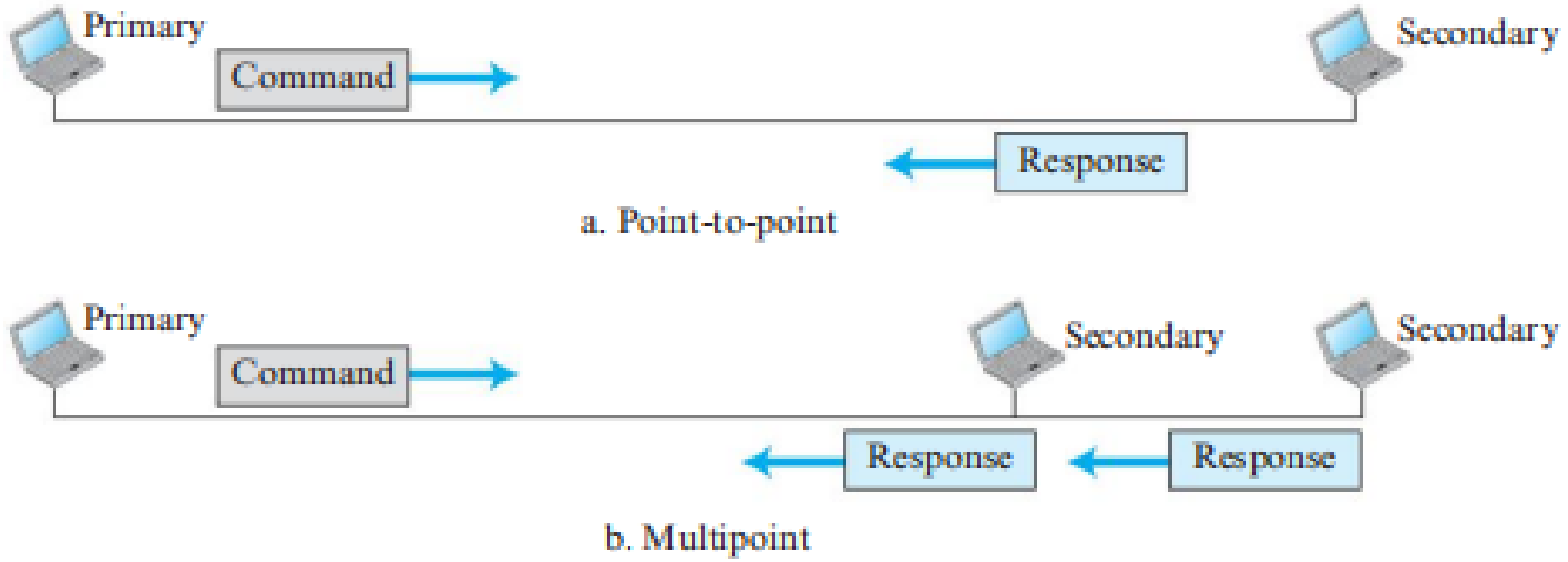
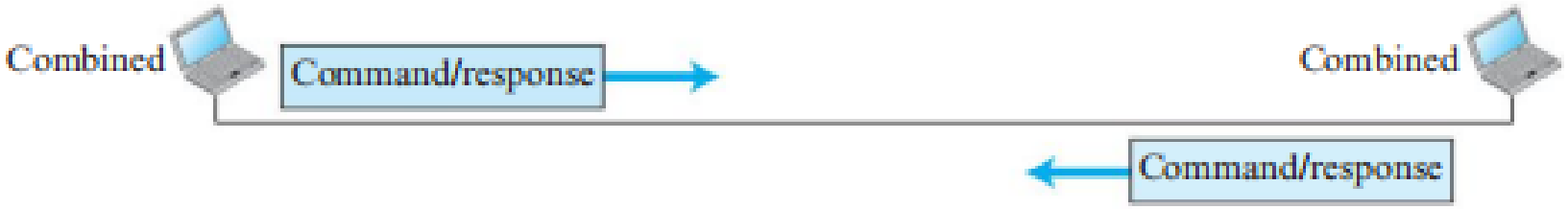


Figure 5.21 *Asynchronous balanced mode*



	NRM	**ABM**
Configuration	Unbalanced	Balanced
Stations	Primary + Secondary	Peer stations
Control	Primary controls	Both communicate
Links	Point-to-point / Multipoint	Point-to-point
Usage	Less common	Common today

Three Types of HDLC Frames

Frame	Main Purpose	Information Field	Control Role
I-Frame	User data transfer	Yes	Can piggyback flow and error-control information
S-Frame	Flow and error control	No	Used when piggybacking is impossible or inappropriate
U-Frame	System and link management	Yes	Carries management information, not primarily user data

I – Information

User data + possible ACK/control information

S – Supervisory

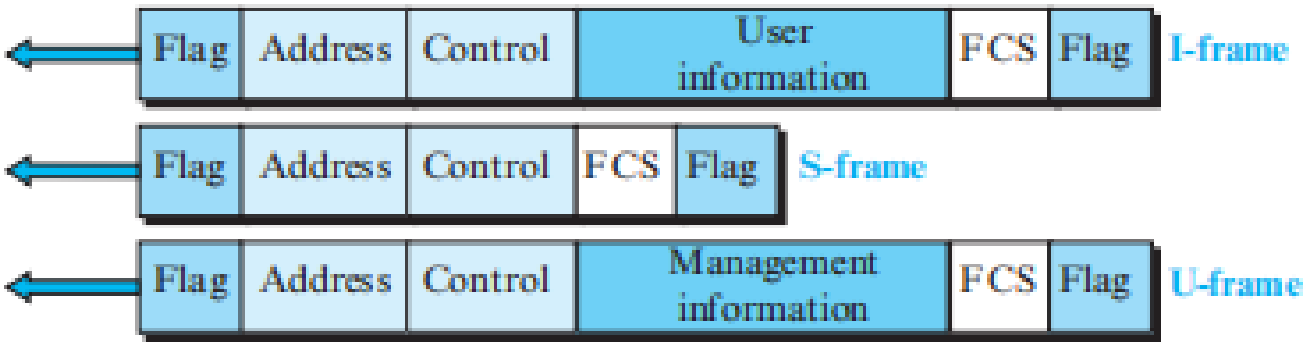
Flow and error control

U – Unnumbered

System and link management

General HDLC Frame Format

Figure 5.22 *HDLC frames*



I-Frame Control Field

$$\boxed{0 \quad N(S) \quad P/F \quad N(R)}$$

N(S): Send Sequence Number

- First bit **0** identifies an I-Frame
- Next 3 bits give sequence numbers **0–7**
- Helps identify frames and reliable transmission

N(R) and P/F

- **N(R):** acknowledgment number during piggybacking
- **P/F = 1:** Poll when Primary → Secondary; Final when Secondary → Primary

Piggybacking

ACK information is carried inside a data frame instead of sending a separate ACK.

S-Frames and Their Control Field

$$\boxed{10 \quad Code \quad P/F \quad N(R)}$$

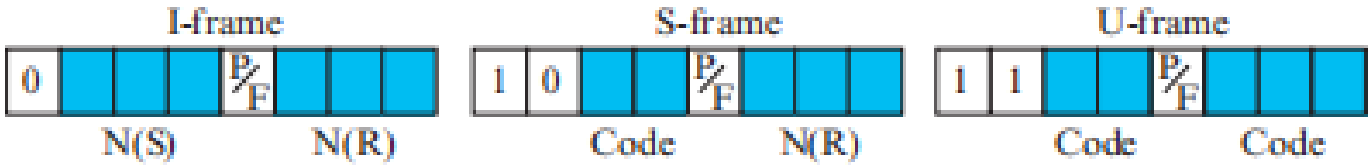
- First two bits **10** identify an S-Frame.
- Used for flow/error control when piggybacking is impossible or inappropriate.
- **Code** selects the S-Frame type; **N(R)** carries ACK/NAK information.
- S-Frames have **no information field**.

Frame	Code	Function
RR	00	ACK: frames received safely; receiver ready
REJ	01	NAK for Go-Back-N; lost/damaged frame reported before timeout
RNR	10	ACK + receiver busy; asks sender to slow down
SREJ	11	NAK for Selective Repeat; retransmit the specific frame

Memory Trick

RR = Ready; RNR = Not Ready; REJ = Reject and resend; SREJ = Selectively resend.

Figure 5.23 *Control field format for the different frame types*



U-Frames: System Management

Purpose

- Exchange session, control and link-management information
- Used between connected devices
- Can contain an information field
- Information is for management, not primarily user data

Control Codes

- 2-bit prefix
- P/F bit
- 3-bit suffix

Prefix + suffix = 5 bits:

$$2^5 = \boxed{32}$$

possible U-Frame types.

HDLC Summary and Exam Focus

HDLC in One View

- Bit-oriented DLC protocol
- Point-to-point and multipoint communication
- Modes: NRM (unbalanced), ABM (balanced)
- Fields: Flag, Address, Control, Information, FCS, Flag

Three Frame Types

- I-Frame: user data + possible piggybacking
- S-Frame: flow and error control
- U-Frame: system/link management

Exam Focus

NRM vs ABM **I vs S vs U Frames** **RR, RNR, REJ, SREJ**

Point-to-Point Protocol (PPP)

What is PPP?

Point-to-Point Protocol (PPP) is a widely used data-link-layer protocol for communication over **point-to-point links**.

- Used to connect two devices directly, traditionally:
  - Home computer $\leftrightarrow$ ISP server
  - Through modem and telephone line
- The telephone line provides **physical-layer services**.
- PPP provides **data-link-layer control and management**.
- PPP is derived from ideas used in **HDLC**.

Key Idea

PPP is designed primarily for **simple point-to-point communication**.

PPP Services: What Does PPP Provide?

Link Services

- Defines PPP frame format.
- Establishes the link.
- Negotiates link options.
- Manages data exchange.
- Terminates the link.

Additional Services

- Supports multiple network-layer protocols.
- Optional authentication.
- Network address configuration.
- Temporary address assignment for users.
- Multilink PPP supports multiple links.

Important

PPP can carry payloads from **several network layers**, not only IP.

PPP Services: What Does PPP NOT Provide?

Services Intentionally Omitted for Simplicity

- **No Flow Control**
  - Sender can transmit several frames continuously.
  - No mechanism prevents receiver overload.
- **Limited Error Control**
  - CRC detects errors.
  - Corrupted frames are silently discarded.
  - Upper-layer protocols handle retransmission.
- **No Sequence Numbering**
  - Packets may potentially arrive out of order.
- **No Sophisticated Multipoint Addressing**
  - PPP is designed for point-to-point links.

Memory Trick

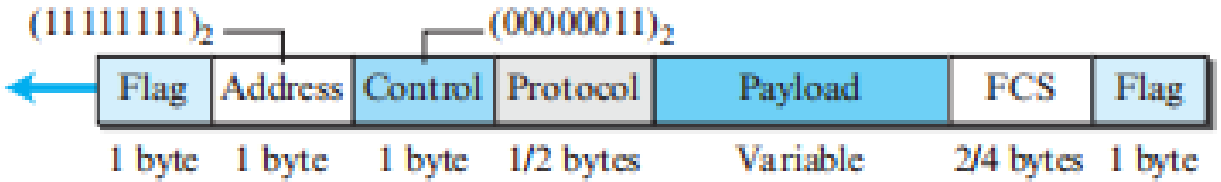
PPP = Simple Link Protocol: Detection, not Recovery

PPP Frame Format

Flag	Address	Control	Protocol	Payload	FCS	Flag

- PPP uses **character-oriented (byte-oriented) framing**.
- Frames begin and end with a **flag**.
- Payload carries:
  - User data, or
  - PPP control information.
- Error detection is performed using **FCS**.

Figure 5.24 *PPP frame format*



PPP Frame Fields

- **Flag**
  - 1 byte.
  - Bit pattern: **01111110**.
  - Marks beginning and end of frame.
- **Address**
  - Constant value: **11111111**.
  - Represents broadcast address.
- **Control**
  - Constant value: **00000011**.
  - Similar to HDLC unnumbered frames.
- **Protocol**
  - Identifies what is carried in payload.
  - Default length: **2 bytes**.
  - Can be reduced to **1 byte** by negotiation.

PPP Payload and FCS

Payload Field

- Carries:
  - User data, or
  - Control information.
- Sequence of bytes.
- Default maximum size: **1500 bytes**.
- Maximum size can be changed during negotiation.
- Byte stuffing is used if the flag pattern appears in data.
- Since there is no explicit length field:
  - Padding may be required for shorter data.

FCS: Frame Check Sequence

- Used for error detection.
- Standard CRC.
- Can be:
  - 2 bytes, or
  - 4 bytes.

PPP Byte Stuffing

Why Byte Stuffing?

PPP is a **byte-oriented protocol**. Therefore, the flag pattern must not accidentally appear inside the data.

- Flag byte:

$$\boxed{01111110}$$

- Escape byte:

$$\boxed{01111101}$$

- If the flag pattern appears inside the data:
  - Insert an **escape byte**.
  - Receiver knows the following byte is data, not a flag.
- If the escape byte itself appears in data:
  - It must also be escaped using another escape byte.

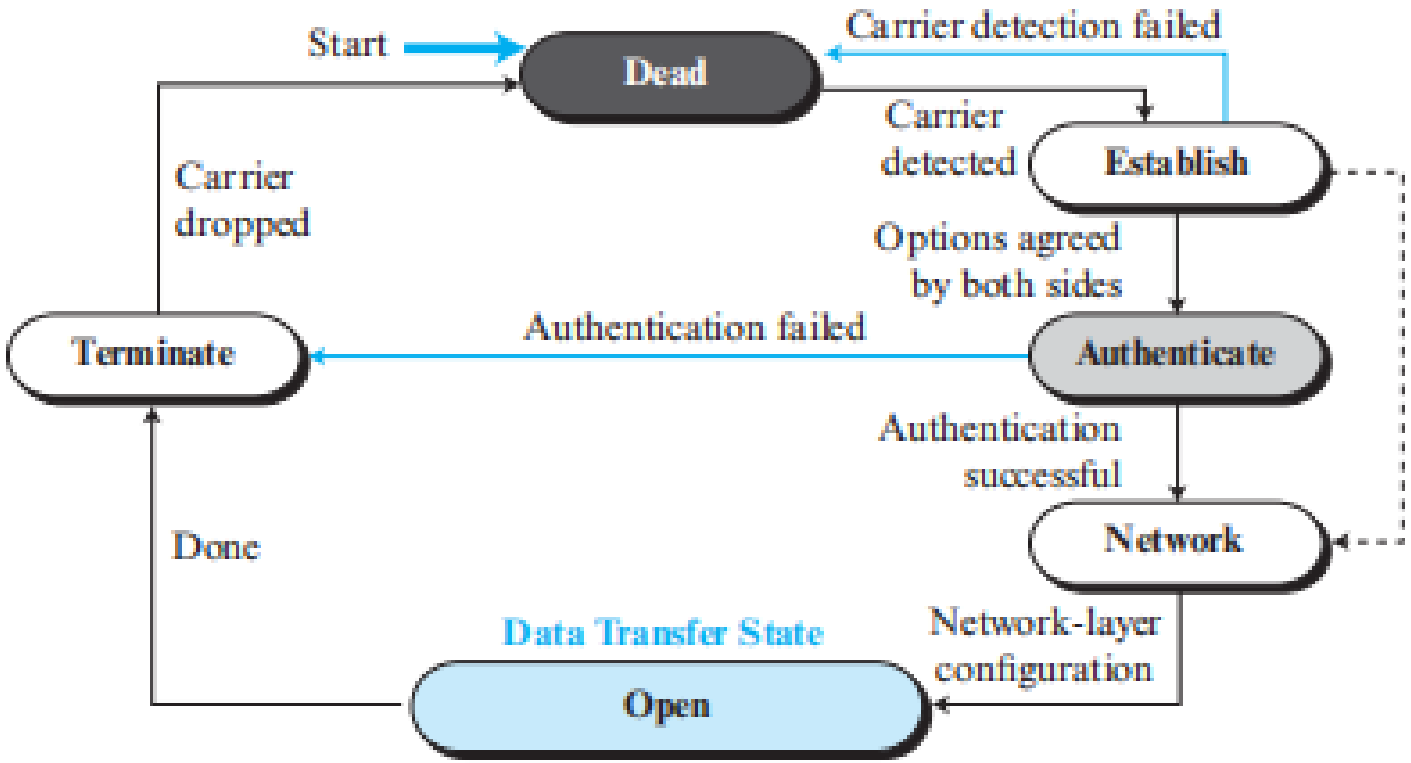
Simple Rule

Special Byte in Data $\Rightarrow$ Escape It

PPP Transition Phases

Dead → Establish → Authenticate → Network → Open → Terminate → Dead

Figure 5.25 *Transition phases*



PPP Transition Phases Explained

- **Dead State**
  - No active physical-layer carrier.
  - Line is quiet.
- **Establish State**
  - One node initiates communication.
  - Link options are negotiated.
- **Authenticate State**
  - Entered if authentication is required.
- **Network State**
  - Network-layer configuration occurs.
- **Open State**
  - Actual data transfer takes place.
- **Terminate State**
  - Connection is being closed.
  - Carrier drops $\Rightarrow$ return to Dead state.

PPP Multiplexing

PPP Uses Multiple Protocol Families

PPP can carry different types of information in its payload.

PPP Payload

↓

Link Control Protocol (LCP)

Authentication Protocols (APs)

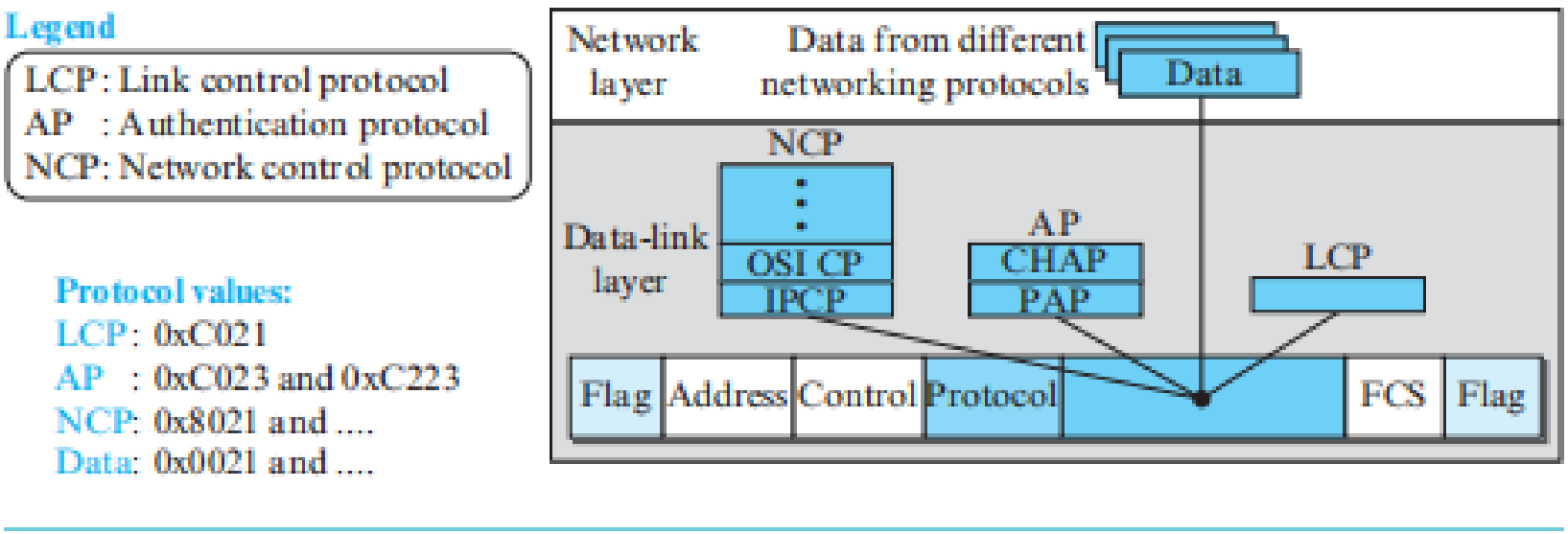
Network Control Protocols (NCPs)

Network-Layer Data

Protocol Sets

- One LCP.
- Two Authentication Protocols.
- Several NCPs.
- Data from multiple network-layer protocols.

Figure 5.26 *Multiplexing in PPP*



Link Control Protocol (LCP)

Role of LCP

The **Link Control Protocol (LCP)** manages the PPP link.

- Establishes the link.
- Maintains the link.
- Configures the link.
- Terminates the link.
- Provides option negotiation.
- Both endpoints must agree on options before:
  - The link can be successfully established.

Easy Recall

LCP = Establish + Configure + Maintain + Terminate

PPP Authentication Protocols

Why Authentication?

PPP was designed for dial-up links where verification of user identity is important.

PAP

Password Authentication Protocol

1. User sends:
   - Username/Identification
   - Password
2. System verifies credentials.
3. Connection:
   - Accepted, or
   - Denied.

CHAP

Challenge Handshake Authentication Protocol

1. System sends challenge.
2. User combines: Challenge Secret password and sends result.
3. System performs same calculation.
4. Results compared:
   - Same $\Rightarrow$ Access granted.
   - Different $\Rightarrow$ Access denied.

PAP vs CHAP

PAP: Simple Authentication

- Two-step procedure.
- Username and password are sent for verification.

CHAP: More Secure Authentication

- Three-way handshake.
- Password is **never sent online**.
- Uses:

$$\text{Challenge} + \text{Secret Password} \rightarrow \text{Response}$$

- System independently calculates the expected result.
- Matching result $\Rightarrow$ access granted.

Key Difference

PAP sends credentials for verification; CHAP keeps the password secret.

Network Control Protocols (NCPs)

Why NCPs?

PPP can carry data from **multiple network-layer protocols**. Each network protocol needs appropriate link configuration.

- PPP defines a specific **Network Control Protocol (NCP)** for each network-layer protocol.
- Examples:
  - **IPCP** → Internet Protocol (IP).
  - Xerox CP → Xerox protocol.
  - Other NCPs → Other network protocols.
- NCP packets:
  - Configure the link for a specific network-layer protocol.
  - Do **not** carry the actual network-layer user data.

Easy Recall

NCP Configures the Network Layer — It Does Not Carry User Data

Network-Layer Data in PPP

After NCP Configuration

Once network-layer configuration is completed, users can exchange actual network-layer packets.

- PPP supports data from different network layers.
- The **Protocol field** identifies the payload type.
- Examples:

IP Data $\Rightarrow (0021)_{16}$

OSI Data $\Rightarrow (0023)_{16}$

PPP Frame

Protocol Field $\Rightarrow$ Identifies Network-Layer Protocol

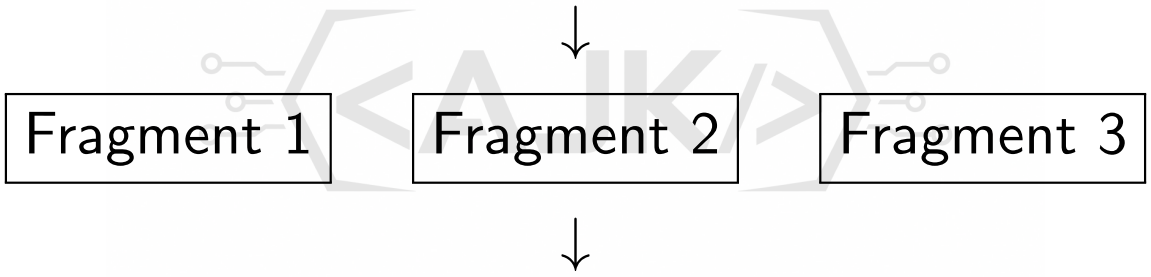
Payload $\Rightarrow$ Carries Actual Network-Layer Data

Multilink PPP

Why Multilink PPP?

Original PPP was designed for a **single-channel point-to-point link**.
Multiple available channels motivated the development of **Multilink PPP**.

One Logical PPP Frame



Multiple Actual PPP Frames

- A logical PPP frame is divided into fragments.
- Each fragment is carried inside an actual PPP frame.
- Protocol field for a fragment:

$$(003d)_{16}$$

Multilink PPP: Additional Requirement

New Complexity

Fragmentation creates an important problem:

How do we know the original position of each fragment?

- A logical PPP frame is divided into multiple fragments.
- Fragments may travel through multiple channels.
- A **sequence number** is added to:
  - Identify fragment position.
  - Reconstruct the original logical PPP frame.

Textbook Figure Banner

Insert Textbook Figure 5.27: Multilink PPP

PPP: Complete Summary

PPP Provides

- Frame format
- Link establishment
- Negotiation
- Authentication
- Network configuration
- Multiprotocol support
- Multilink capability
- CRC error detection

PPP Does Not Provide

- Flow control
- Sophisticated error recovery
- Sequence numbering
- Multipoint addressing

PPP Architecture

LCP → Authentication → NCP → Network Data

PPP: Key Takeaways

1. PPP is a **data-link-layer protocol** for point-to-point links.
2. It uses **byte-oriented framing** and byte stuffing.
3. PPP provides **link establishment and configuration**.
4. It uses **CRC** for error detection but does not provide sophisticated error recovery.
5. PPP provides optional authentication using:
   - PAP
   - CHAP
6. **LCP** manages the link.
7. **NCP** configures the network layer.
8. PPP supports multiple network-layer protocols.
9. **Multilink PPP** divides one logical frame into multiple actual frames.

Final Memory Formula

PPP = Frame + Link Control + Authentication + Network Configuration